

UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

GYMPRO

Plataforma de Gestión para Gimnasios

PIPELINE DE DATOS

Recolección • Limpieza • Transformación • Análisis
Visualización • Extracción • Toma de Decisiones

MATERIA:

EXTRACCIÓN DE CONOCIMIENTO EN BASES DE DATOS

INTEGRANTES:

MAXIMILIANO ARRIAGA MORA

MIGUEL ANGEL HERNANDEZ CERVANTES

JESUS FRANCISCO PEREZ MEDINA

JUAN CARLOS PEREZ NAVA

PROFESOR:

MGTI HÉCTOR VELÁZQUEZ ESTRADA

GRUPO:

IGDS - 93

Índice de Contenidos

Índice de Contenidos	2
1. Introducción	4
1.1 Arquitectura General del Pipeline	4
2. Recolección de Datos	5
2.1 Fuentes de Datos Operativas	5
2.1.1 Membresías y Pagos	5
2.1.2 Asistencia	5
2.1.3 Progreso Físico	5
2.1.4 Rutinas y Ejercicios	5
2.1.5 Datos de la Plataforma SaaS	5
2.2 Validación en el Punto de Entrada	5
3. Limpieza de Datos	7
3.1 Normalización de Fechas	7
3.2 Filtrado de Nulos y Valores Inválidos	7
3.3 Función <code>_safe_float()</code>	7
3.4 Homogeneización de Métodos de Pago	7
4. Transformación de Datos	8
4.1 Aggregation Pipelines de MongoDB	8
4.2 Ingeniería de Features para K-Means	8
4.3 Ingeniería de Features para Predicción de Cancelaciones	8
4.4 Construcción de Series Temporales	8
5. Análisis de Datos	10
5.1 MapReduce de Ingresos y Asistencia	10
5.2 Clustering K-Means de Composición Corporal	10
5.3 Regresión Lineal (Ridge) para Predicción de Peso	10
5.4 Clasificación de Riesgo de Cancelación	10
5.5 Caché de Resultados	10
6. Visualización de Datos	11
6.1 Dashboard de KPIs Operativos	11
6.2 Analytics MapReduce	11
6.3 Analytics K-Means	11
6.4 Regresión y Cancelaciones	11
6.5 Dashboard del Miembro	11
7. Extracción de Datos	12
7.1 Exportación CSV	12
7.2 Exportación PDF	12

7.3 Sistema de Respaldos	12
8. Toma de Decisiones	13
8.1 Decisiones por Rol	13
8.2 Alertas y Notificaciones.....	13
8.3 Ciclo de Retroalimentación	13
8.4 Limitaciones y Trabajo Futuro	13
9. Conclusiones	15

1. Introducción

GymPro es una plataforma SaaS multi-tenant diseñada para la gestión integral de gimnasios. Sobre una arquitectura Flask + MongoDB + PostgreSQL, el sistema implementa un pipeline de datos completo que abarca desde la captura de eventos operativos hasta la generación de inteligencia de negocio mediante modelos de machine learning.

El presente documento describe en detalle cada etapa de ese pipeline: cómo se originan los datos, cómo se validan y limpian, qué transformaciones se aplican, qué modelos analíticos se ejecutan, cómo se visualizan los resultados, de qué formas se exportan y, finalmente, cómo orientan la toma de decisiones estratégicas.

1.1 Arquitectura General del Pipeline

Capa	Responsabilidad
Colección	Registro de eventos operativos vía API REST (Flask + Gunicorn)
Almacenamiento	MongoDB 7 (documentos flexibles) + PostgreSQL 16 (entidades estructuradas)
Análisis	scikit-learn en proceso: K-Means, Ridge Regression, RandomForestClassifier
Caché	Redis con TTL configurable por resultado analítico (ANALYTICS_CACHE_TTL_HOURS)
Presentación	React + Recharts (web) + Expo / React Native (móvil)
Exportación	CSV en memoria + PDF con ReportLab bajo demanda

2. Recolección de Datos

GymPro capta datos de manera continua a través de cuatro canales: interacción de usuarios con la interfaz web, entrada manual por personal del gimnasio (repcionistas, entrenadores), registros automáticos de pagos y membresías, y mediciones físicas periódicas de los miembros.

2.1 Fuentes de Datos Operativas

2.1.1 Membresías y Pagos

Cada transacción económica se persiste en la colección MongoDB pagos con los campos `id_miembro`, `id_gimnasio`, `monto`, `metodo_pago` y `fecha_pago`. El modelo `Pago` (`models/pago.py`) normaliza la fecha a formato ISO 8601 en la inserción, garantizando homogeneidad independientemente del cliente que envía el request.

- Endpoint POST `/api/pagos`: registra un pago y vincula automáticamente al miembro activo.
- Endpoint POST `/api/miembro_membresias`: activa o renueva una membresía con fecha de vencimiento calculada.
- Integración Stripe (`billing_stripe.py`): maneja pagos recurrentes y webhooks de eventos de facturación.

2.1.2 Asistencia

El registro de entrada/salida se captura en la colección `asistencias`. Cada documento contiene `id_miembro`, `fecha` (datetime normalizado a 00:00:00 UTC para facilitar búsquedas por día), `hora_entrada` y `hora_salida` almacenados como strings HH:MM:SS.

- Endpoint POST `/api/asistencia/entrada`: registra hora de llegada del miembro.
- Endpoint PUT `/api/asistencia/salida`: cierra el registro con `hora_salida`.

2.1.3 Progreso Físico

Entrenadores y miembros registran mediciones corporales periódicas: peso, IMC, grasa corporal, masa muscular, agua corporal, masa ósea y diez circunferencias (cintura, cadera, pecho, brazos, muslos, pantorrilla). El método `calcular_bmi()` computa el Índice de Masa Corporal a partir de peso y estatura en el mismo request de creación.

- Colección MongoDB: `progreso_fisico`.
- Endpoint POST `/api/progreso`: inserta nueva entrada de medición del miembro.

2.1.4 Rutinas y Ejercicios

El sistema gestiona rutinas asignadas por entrenadores (colección `rutinas` en MongoDB) y el catálogo de ejercicios en PostgreSQL (tabla `ejercicio`). Cada rutina almacena series, repeticiones, peso objetivo y video demostrativo de hasta 30 segundos.

2.1.5 Datos de la Plataforma SaaS

Las entidades de plataforma — gimnasios, suscripciones, planes y facturación — residen en PostgreSQL. La separación refleja la naturaleza distinta de los datos: documentos flexibles (MongoDB) para el core operativo y datos relacionales rígidos (PostgreSQL) para gobernanza multi-tenant.

2.2 Validación en el Punto de Entrada

Cada endpoint aplica validaciones antes de persistir: tipos de dato, rangos lógicos (peso > 0, monto > 0), existencia del miembro y del gimnasio. El helper `require_tenant()` inyecta `id_gimnasio` en todos los filtros de escritura, garantizando aislamiento entre tenants.

3. Limpieza de Datos

La limpieza en GymPro opera en dos momentos: en el ingreso (validaciones en el modelo de dominio) y en el análisis (normalización previa a los algoritmos de ML). Esta estrategia evita propagar datos corruptos al pipeline analítico.

3.1 Normalización de Fechas

El campo `fecha_pago` puede llegar como string ISO, objeto `datetime` o valor nulo según el cliente que origina el request. La función `_to_naive_datetime()` en `spark_regresion.py` resuelve esta heterogeneidad con un esquema de coerción por prioridad:

- Si el valor es un objeto `datetime`, se elimina `tzinfo` para uniformidad.
- Si es un objeto `date`, se convierte a `datetime(year, month, day)`.
- Si es string, se intenta con múltiples formatos: ISO completo, solo fecha, con hora.
- Si ningún formato aplica, el registro se descarta del análisis sin propagación de error.

En el pipeline MapReduce (`spark_mapreduce.py`) se usa la expresión `$toDate` de MongoDB para normalizar en el lado del motor de base de datos, reduciendo el volumen de datos que llega a Python.

3.2 Filtrado de Nulos y Valores Inválidos

El clustering K-Means requiere features completos. `spark_kmeans.py` filtra miembros con `peso_inicial` nulo o cero, y `estatura` nula o cero (previene división por cero en el cálculo de IMC). Para registros sin progreso físico reciente se aplican fallbacks razonables: `grasa` = 20.0%, `musculo` = 30.0%, que ubican al miembro en rangos promedio sin distorsionar el clustering.

3.3 Función `_safe_float()`

Todos los valores monetarios pasan por `_safe_float()` antes de serializarse a JSON. Esta función captura `TypeError` y `ValueError` devolviendo 0.0, y aplica redondeo a dos decimales para evitar imprecisiones de punto flotante en la presentación.

3.4 Homogeneización de Métodos de Pago

El campo `metodo_pago` es texto libre en MongoDB. El pipeline MapReduce sustituye valores `None` o ausentes por la etiqueta 'Sin especificar', manteniendo la cohesión de las agrupaciones sin perder registros.

4. Transformación de Datos

La capa de transformación convierte datos crudos en estructuras optimizadas para análisis y presentación. GymPro aplica transformaciones tanto en MongoDB (aggregation pipelines) como en Python (scikit-learn preprocessing).

4.1 Aggregation Pipelines de MongoDB

El patrón MapReduce de ingresos se implementa como una cadena de etapas de aggregation equivalente a un groupBy + aggregate de Spark, pero ejecutado directamente en el motor de base de datos:

- `$match`: filtra por `gym_id` (aislamiento tenant) y descarta fechas nulas.
- `$addFields` + `$dateToString`: extrae el período YYYY-MM de `fecha_pago`, normalizando strings e `ISODate` por igual.
- `$group`: agrupa por (periodo, metodo_pago) calculando suma de ingresos, conteo de transacciones y promedio.
- `$sort`: ordena cronológicamente para renderizado directo en gráficas de serie temporal.

El pipeline de asistencia sigue la misma estructura, agrupando por (`dia_semana`, `hora_bloque`) para construir el mapa de calor de afluencia.

4.2 Ingeniería de Features para K-Means

El clustering de miembros requiere una matriz de features numéricos con escala uniforme. El proceso combina la colección miembros con el último registro de progreso_fisico usando `$group` con `$first`, construye el vector [peso, IMC, grasa_corporal, masa_muscular] por miembro, y aplica `StandardScaler` para transformar cada feature a media=0 y desviación=1, garantizando que ninguna variable domine las distancias euclideas.

4.3 Ingeniería de Features para Predicción de Cancelaciones

El modelo `RandomForestClassifier` opera sobre cinco features derivados de varias colecciones:

Feature	Descripción y Derivación
dias_sin_asistir	Diferencia entre <code>now()</code> y la última fecha de asistencia (<code>\$max</code> sobre colección asistencias).
num_asistencias_ult60	Conteo de asistencias en los últimos 60 días usando <code>\$cond</code> + <code>\$gte</code> en la aggregation.
total_pagos	Número histórico de pagos del miembro (<code>\$count</code> sobre colección pagos).
meses_activo	Diferencia en meses entre <code>now()</code> y <code>fecha_registro</code> del miembro.
tiene_membresia_activa	1.0 si la membresía está vigente, 0.0 si venció (colección <code>miembro_membresia</code>).

4.4 Construcción de Series Temporales

El dashboard KPI calcula ingresos de los últimos 6 meses mediante `_prev_months()`, que genera tuplas (year, month) en orden cronológico ascendente. Los resultados del pipeline

MongoDB se mapean con `get(ym, 0.0)` para garantizar que meses sin transacciones aparezcan como cero y no como huecos en la gráfica.

5. Análisis de Datos

El módulo de análisis implementa cuatro técnicas de machine learning y análisis estadístico, todas ejecutadas en proceso sin JVM ni servicios externos, con resultados cacheados en Redis para garantizar latencia sub-100ms en requests subsecuentes.

5.1 MapReduce de Ingresos y Asistencia

Implementado en `app/routes/ia/spark_mapreduce.py`. Emula el paradigma Map→Reduce usando MongoDB aggregation pipelines nativos. Genera dos artefactos: detalle por período y método de pago (para análisis de canal de ingreso) y resumen mensual (base de las gráficas de tendencia). El mapa de asistencia produce una matriz hora x día de la semana que revela picos de afluencia para optimizar staffing.

5.2 Clustering K-Means de Composición Corporal

Implementado en `app/routes/ia/spark_kmeans.py`. Aplica K-Means (scikit-learn) con `k` configurable (default 3) sobre la matriz estandarizada de composición corporal.

- Métrica de calidad: Silhouette Score — valores ≥ 0.5 indican clusters bien separados.
- Etiquetas semánticas: cada cluster recibe un label del pool predefinido, asignado por ranking de IMC promedio.
- Centroides: retornados desnormalizados (inversa de StandardScaler) para interpretación directa por el entrenador.

5.3 Regresión Lineal (Ridge) para Predicción de Peso

Implementado en `app/routes/ia/spark_regresion.py`. Entrena un modelo Ridge Regression sobre la serie temporal de `progreso_fisico` del gimnasio. Variables predictoras: días desde el registro, cintura (cm) y grasa corporal (%). Variable objetivo: peso corporal actual (kg). Métricas reportadas: R^2 , RMSE y MAE. La predicción individual usa coeficientes cacheados sin re-entrenar, con latencia objetivo $< 100ms$.

5.4 Clasificación de Riesgo de Cancelación

Implementado en `app/routes/ia/spark_cancelaciones.py`. RandomForestClassifier que predice la probabilidad de baja de membresía para cada miembro activo.

- Label de entrenamiento: 1 = en riesgo (> 21 días sin asistir O membresía vencida); 0 = activo estable.
- Salida por miembro: probabilidad de cancelación (0–1) y clasificación en banda Bajo / Medio / Alto.
- Métrica: ROC-AUC y Accuracy sobre el conjunto de entrenamiento.

5.5 Caché de Resultados

`spark_config.py` implementa `cache_get()` / `cache_set()` sobre Redis con TTL configurable (ANALYTICS_CACHE_TTL_HOURS, default 24h). Las claves siguen el formato `mapreduce_gym{id}` y `kmeans_gym{id}_k{k}` para soporte multi-tenant. El patrón es read-through: si el caché falla, se re-ejecuta el análisis y se re-cachea.

6. Visualización de Datos

La capa de presentación está implementada en React usando Recharts para gráficas interactivas. Cada rol del sistema (SuperAdmin, Admin, Owner, Entrenador, Miembro, Recepcionista) accede a un conjunto diferenciado de visualizaciones según su alcance operativo.

6.1 Dashboard de KPIs Operativos

Dashboard.jsx (roles admin y owner_gym) consume el endpoint /api/dashboard/kpis y presenta: miembros activos en tiempo real filtrados por tenant, ingresos del mes actual, y tendencia de ingresos de los últimos 6 meses en BarChart con CustomTooltip temático.

6.2 Analytics MapReduce

AnalyticsMapReduce.jsx presenta tres secciones de gráficas: LineChart de ingresos por período (evolución mensual), BarChart por método de pago (volumen por canal), y PieChart de distribución proporcional. Todas las gráficas comparten el componente CustomTooltip con tema oscuro coherente con el diseño del sistema.

6.3 Analytics K-Means

AnalyticsKMeans.jsx permite ajustar el parámetro k dinámicamente y visualiza: tarjetas por cluster con nombre del perfil, tamaño del grupo e indicadores de composición corporal promedio. El color codifica riesgo semántico (rojo = alta prioridad, amarillo = intermedio, verde = óptimo). El Silhouette Score se presenta como badge con calificación verbal (Excelente / Bueno / Aceptable / Bajo). Una tabla paginada lista las asignaciones individuales.

6.4 Regresión y Cancelaciones

- Regresión (AnalyticsRegresion.jsx): coeficientes del modelo, métricas R^2 /RMSE, y predicción de peso futuro por miembro individual con selector de horizonte temporal.
- Cancelaciones: lista priorizada de miembros con mayor probabilidad de baja, banda de riesgo codificada por color, días sin asistir y asistencias en los últimos 60 días.

6.5 Dashboard del Miembro

UserDashboard.jsx presenta al miembro su propia curva de progreso físico (peso, IMC, grasa corporal) en LineCharts de serie temporal, junto con el estado de su membresía activa, historial de pagos y próximos vencimientos.

7. Extracción de Datos

GymPro provee mecanismos de extracción en dos formatos estándar (CSV y PDF) y un sistema de respaldos completos de base de datos para recuperación ante desastres.

7.1 Exportación CSV

El módulo `app/routes/admin/reports.py` implementa `GET /api/reports/{tipo}/csv` para cuatro tipos de reporte:

Tipo	Contenido del CSV
mapreduce	Período, método de pago, ingresos totales, número de pagos, promedio por transacción.
kmeans	ID miembro, nombre, cluster asignado, perfil, peso, IMC, grasa corporal, masa muscular.
miembros	Nombre, email, sexo, estado, objetivo, peso inicial, estatura, fecha de registro.
cancelaciones	Nombre, probabilidad de cancelación, días sin asistir, asistencias recientes, banda de riesgo.

La respuesta usa `mime-type text/csv; charset=utf-8` con `Content-Disposition: attachment`, forzando descarga directa desde el navegador sin renderizado intermedio.

7.2 Exportación PDF

`GET /api/reports/{tipo}/pdf` genera documentos PDF en memoria usando ReportLab (SimpleDocTemplate, landscape A4). El helper `_pdf_response()` encapsula la construcción del documento garantizando que la respuesta sea un buffer en memoria sin escritura a disco. Los PDFs incluyen encabezado con el nombre del reporte, fecha de generación y tabla formateada con la paleta visual de GymPro.

7.3 Sistema de Respaldos

El módulo `backups (app/backups/)` implementa un pipeline completo de backup y restore:

- Backup MongoDB: `mongodump` comprimido en `.tar.gz`, almacenado en `/app/storage/backups/` (volumen Docker persistente mapeado a `./backups` en el host).
- Backup PostgreSQL: `pg_dump` en formato custom (`-Fc`) con parámetros leídos de variables de entorno, habilitando restore selectivo de tablas.
- Tenant backups: `tenant_service.py` extrae únicamente los documentos del `id_gimnasio` solicitante para respaldos aislados por cliente.
- Restore: `restore_service.py` implementa restauración incremental gestionable desde `BackupsDashboard.jsx`.

8. Toma de Decisiones

El pipeline está diseñado para cerrar el ciclo entre dato y acción. Cada capa analítica produce salidas concretas que habilitan decisiones operativas y estratégicas en los distintos roles del sistema.

8.1 Decisiones por Rol

Rol	Decisiones habilitadas por el pipeline
Administrador / Owner	Ajustar staffing según mapa de afluencia por hora y día. Identificar métodos de pago preferidos. Lanzar campañas de retención en meses de baja actividad.
Entrenador	Personalizar rutinas según cluster de composición corporal. Ver predicción de peso a 30/60/90 días. Priorizar seguimiento a miembros con alta probabilidad de abandono.
Recepcionista	Visualizar alertas de vencimiento de membresía en el check-in. Identificar miembros en riesgo para notificación proactiva.
Miembro	Monitorear su propia curva de progreso físico vs. objetivos declarados. Consultar historial de pagos y estado de membresía.
SuperAdmin (plataforma)	Supervisar salud global: gimnasios activos, suscripciones vencidas, facturación Stripe pendiente.

8.2 Alertas y Notificaciones

El módulo de notificaciones implementa alertas basadas en umbrales derivados del pipeline analítico:

- Riesgo de cancelación alto (probabilidad > 0.7): genera notificación para el administrador con lista de miembros prioritarios.
- Membresía próxima a vencer (< 7 días): alerta al recepcionista para contacto proactivo.
- Inactividad prolongada (> 21 días sin asistir): activa el label en riesgo del clasificador de cancelaciones.

8.3 Ciclo de Retroalimentación

Los datos generados por las decisiones (nuevos pagos, asistencias, actualizaciones de progreso) retroalimentan el pipeline en el siguiente ciclo de análisis. El TTL del caché Redis garantiza re-entrenamiento automático cada 24 horas. El botón 'Entrenar modelo' en el dashboard dispara POST `/api/analytics/{tipo}/train` para actualización inmediata bajo demanda.

8.4 Limitaciones y Trabajo Futuro

El pipeline actual opera completamente en proceso, adecuado hasta aproximadamente 100,000 registros por gimnasio. Las mejoras previstas para mayor escala incluyen:

- Reemplazar el entrenamiento en proceso por trabajos batch con Celery + Redis como broker.

- Implementar validación cruzada (k-fold) en el modelo de cancelaciones para métricas fuera de muestra más confiables.
- Añadir monitorización de drift de modelos para re-entrenar automáticamente cuando la distribución de features cambie significativamente.
- Agregar pipeline ETL hacia un data warehouse (BigQuery / Redshift) para análisis histórico sin impactar la base de datos operativa.

9. Conclusiones

GymPro implementa un pipeline de datos completo y funcional sobre una arquitectura moderna y escalable. Desde la captura de eventos operativos hasta la ejecución de modelos de machine learning y la entrega de visualizaciones interactivas, cada etapa está diseñada con criterios de aislamiento multi-tenant (todas las consultas filtran por `id_gimnasio`), bajo acoplamiento (cada módulo de análisis es independiente y cacheado), y extensibilidad (nuevos modelos se agregan como blueprints Flask sin modificar el core).

El resultado es un sistema que convierte los datos operativos del día a día de un gimnasio en inteligencia accionable: desde identificar los horarios de mayor afluencia hasta predecir qué miembros están a punto de cancelar su membresía, entregando esa información en el momento y contexto adecuados para cada rol del sistema.